

Déroulement Détaillé des Appels

Protocoles de sécurité · Paramètres · Réponses · Implémentation Java

Source doc	developer.bankid.cz/docs/security_sep (v1.3 – 10.04.2025)
Certificat serveur	X.509 FQDN standard (QWAC non requis — SHOULD uniquement)
TLS	1.3 obligatoire (MUST depuis 01/07/2025)
Auth method	/token : client_secret_post (SeP standard)
JWS/JWE	MAY — code gère les deux cas (JSON brut et JWT)

0. Certificat Serveur — Pourquoi pas QWAC

La doc **security_sep** indique : *"We RECOMMEND the use of QWAC certificates for server authentication."* C'est un **SHOULD**, pas un **MUST**. Un certificat X.509 commercial standard est parfaitement conforme, à condition de respecter les contraintes **MUST** ci-dessous.

Contrainte	Niveau	Ce que ça signifie pour toi
FQDN obligatoire	MUST	kyc.ton-domaine.com — pas *.ton-domaine.com
Pas de wildcard	MUST NOT	Certificat dédié par FQDN uniquement
CA de confiance	MUST	DigiCert, Let's Encrypt, Sectigo... — toute CA reconnue
ECDSA $\geq$ 256 bits ou RSA $\geq$ 2048 bits	MUST	Taille de clé minimale
Algorithme de hash sécurisé	MUST	SHA-256 minimum — MD5 et SHA1 interdits
Rotation annuelle	MUST	Renouvellement au moins une fois par an
TLS 1.3 uniquement	MUST	Forcer TLS 1.3, interdire la renégociation vers 1.2
QWAC	SHOULD	Recommandé mais non obligatoire — standard suffit

✓ Ce que tu utilises à la place du QWAC

Un certificat TLS standard émis par Let's Encrypt (gratuit) ou DigiCert/Sectigo (payant).

Configuré sur la Gateway Spring Cloud Gateway avec FQDN exact (ex: kyc.mondomaine.com).

Renouvellement automatique possible avec Let's Encrypt + Certbot ou ACME4J.

Aucune démarche auprès d'une CA qualifiée EU — c'est le principal avantage.

```
# application.yml – Gateway avec certificat standard (pas QWAC)
server:
  ssl:
    enabled: true
    enabled-protocols: TLSv1.3    # MUST : TLS 1.3 uniquement
    key-store: ${GATEWAY_KEYSTORE_PATH}    # PKCS12 contenant clé privée + cert FQDN
    key-store-password: ${GATEWAY_KS_PWD}
    key-store-type: PKCS12
    key-alias: kyc-gateway
    # Trust store BankID (CA root + intermédiaires fournis dans la doc)
    trust-store: ${BANKID_TRUSTSTORE_PATH}
    trust-store-password: ${BANKID_TS_PWD}
    # JAMAIS de wildcard : 'kyc.mondomaine.com' et pas '*.mondomaine.com'
```

1. Vue d'Ensemble — Les 6 Appels du Flux KYC

Le flux complet d'une authentification KYC comprend 6 appels distincts. Trois impliquent le navigateur de l'utilisateur (redirections), trois sont des appels serveur-à-serveur depuis ton Back API vers BankID.

#	Appel	Type	Qui appelle	TLS 1.3	JWS/JWE
①	GET /auth → BankID	Redirect navigateur	Navigateur → BankID	MUST	Non
②	Callback /callback?code=...	Redirect navigateur	BankID → Navigateur → Gateway	MUST	Non
③	POST /token	Serveur → Serveur	Back API → BankID	MUST	id_token MAY
④	GET /userinfo ou /profile	Serveur → Serveur	Back API → BankID	MUST	Réponse MAY
⑤	POST /revoke (fin session)	Serveur → Serveur	Back API → BankID	MUST	Non
⑥	GET /.well-known/jwks (startup)	Serveur → Serveur	Back API → BankID	MUST	Non

■ Flux complet résumé

- ① Navigateur → Gateway → redirect BankID /auth
- ② BankID → redirect navigateur → Gateway /callback?code=XXX&state;=YYY
- ③ Back API → BankID POST /token (échange code → access_token + id_token + refresh_token)
- ④ Back API → BankID GET /userinfo (données KYC avec access_token)
→ Stockage fichier chiffré AES-256-GCM + audit
- ⑤ Back API → BankID POST /revoke (fin de session ou expiration)
- ⑥ Back API → BankID GET /.well-known/jwks (au démarrage, cache 10 min)

①

Redirection vers BankID — GET /auth

Construction de l'URL d'autorisation

2.1 Rôle

Ton application redirige le navigateur de l'utilisateur vers l'endpoint /auth de BankID. C'est le point de départ du flux OAuth2 Authorization Code. L'utilisateur s'authentifie auprès de sa banque et consent à partager ses données.

2.2 Paramètres de la requête

Paramètre	Obligatoire	Valeur / Description	Sécurité
client_id	MUST	Ton identifiant BankID (portail développeur)	Identifie ton application
redirect_uri	MUST	https://kyc.tondomaine.com/callback	Doit être whitelisée sur le portail
response_type	MUST	code	Authorization Code flow uniquement
scope	MUST	openid + scopes KYC souhaités	'openid' toujours requis
state	MUST	UUID aléatoire cryptographique	Anti-CSRF : vérifier au callback
nonce	MUST	UUID aléatoire cryptographique	Anti-replay : vérifié dans l'id_token
prompt	SHOULD	login	Force la ré-authentification bancaire
acr_values	SHOULD	loa3	Niveau d'assurance élevé pour KYC

2.3 Sécurité du state et du nonce

MUST

state et nonce DOIVENT être des valeurs aléatoires cryptographiques (SecureRandom), uniques par requête, stockées en session.

MUST

Le state reçu au callback DOIT être identique au state envoyé. Toute différence = rejet immédiat (CSRF).

MUST

Le nonce envoyé DOIT être vérifié dans le claim 'nonce' de l'id_token reçu au /token.

2.4 Implémentation

```
@Service
public class BankIdAuthService {

    public String buildAndStoreAuthUrl(HttpSession session) {
        // Génération cryptographique – MUST : SecureRandom
        String state = generateSecureRandom(); // UUID v4 via SecureRandom
        String nonce = generateSecureRandom();

        // Stockage en session – durée max 30 minutes
        session.setAttribute("bankid_state", state);
        session.setAttribute("bankid_nonce", nonce);
        session.setMaxInactiveInterval(1800); // 30 min

        return UriComponentsBuilder.fromHttpUrl(bankIdBaseUrl + "/auth")
            .queryParams("client_id", clientId)
            .queryParams("redirect_uri", redirectUri)
            .queryParams("response_type", "code")
            .queryParams("scope",
                "openid name birthdate addresses idcards legalstatus offline_access")
            .queryParams("state", state)
            .queryParams("nonce", nonce)
            .queryParams("prompt", "login")
            .queryParams("acr_values", "loa3")
            .build().toUriString();
    }

    private String generateSecureRandom() {
        byte[] bytes = new byte[32];
        new SecureRandom().nextBytes(bytes);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
    }
}
```

2.5 Ce que reçoit BankID / Ce que voit l'utilisateur

BankID affiche une page d'authentification. L'utilisateur choisit sa banque, s'authentifie (application bancaire, SMS OTP...), puis voit la liste des données qu'il autorise à partager (selon les scopes demandés). Il peut accepter ou refuser.

②

Callback BankID — GET /callback

Réception du code d'autorisation

3.1 Rôle

Après l'authentification et le consentement (ou le refus), BankID redirige le navigateur de l'utilisateur vers ton `redirect_uri`. Ce n'est PAS un appel direct de BankID vers ton serveur — c'est le navigateur qui transporte le code.

3.2 Paramètres reçus — cas succès

Paramètre	Type	Description	Sécurité
code	String	Code d'autorisation à usage unique, valide ~60 secondes	Ne jamais logger en clair
state	String	Doit correspondre exactement au state envoyé à /auth	Vérifier IMPÉRATIVEMENT
session_state	String (opt)	Identifiant de session BankID (informatif)	Non critique

3.3 Paramètres reçus — cas erreur

Paramètre	Description
error	Code erreur OAuth2 (access_denied, eid_doesnt_exist, invalid_scope...)
error_description	Description lisible de l'erreur
state	Toujours présent — vérifier même en cas d'erreur

3.4 Traitement obligatoire

MUST Vérifier la présence du paramètre 'error' EN PREMIER — avant tout traitement du code.

MUST Comparer state reçu == state en session. Si différent : rejeter et logger l'incident de sécurité.

MUST Le code est à usage unique — ne JAMAIS le réutiliser ni le cacher côté client.

3.5 Implémentation

```
@GetMapping("/callback")
public ResponseEntity<Void> handleCallback(
    @RequestParam(required = false) String code,
    @RequestParam(required = false) String state,
    @RequestParam(required = false) String error,
    @RequestParam(required = false) String error_description,
    HttpSession session) {

    // ÉTAPE 1 : Vérifier l'erreur EN PREMIER
    if (error != null) {
        log.warn("BankID callback error: {} - {}", error, error_description);
        return switch (error) {
            case "access_denied" -> redirect("/kyc/refused");
            case "eid_doesnt_exist" -> redirect("/kyc/not-eligible");
            default -> redirect("/kyc/error?code=" + error);
        };
    }

    // ÉTAPE 2 : Vérification anti-CSRF du state – MUST
    String expectedState = (String) session.getAttribute("bankid_state");
    if (expectedState == null || !MessageDigest.isEqual(
        expectedState.getBytes(), state.getBytes())) {
        log.error("SECURITY: State mismatch – possible CSRF attack");
        throw new SecurityException("State mismatch");
    }
    session.removeAttribute("bankid_state"); // Invalider après usage

    // ÉTAPE 3 : Orchestration (appels ③ et ④)
    String subHash = kycOrchestrator.processCode(code,
        (String) session.getAttribute("bankid_nonce"));
    session.removeAttribute("bankid_nonce");

    return redirect("/kyc/success?ref=" + subHash);
}
```

③

Échange du code — POST /token

Appel serveur-à-serveur vers BankID

4.1 Rôle

Ton Back API échange le code reçu au callback contre des tokens. Cet appel est **serveur-à-serveur** — jamais depuis le navigateur. La méthode d'authentification est **client_secret_post** (SeP standard).

4.2 Requête HTTP

Élément	Valeur
Méthode	POST
URL	https://oidc.sandbox.bankid.cz/token
Content-Type	application/x-www-form-urlencoded
TLS	1.3 obligatoire — connexion WebClient configurée avec TLSv1.3
Authentification	client_secret_post (client_id + client_secret dans le body)

4.3 Paramètres du body

Paramètre	Obligatoire	Valeur
grant_type	MUST	authorization_code
code	MUST	Le code reçu au callback (usage unique)
redirect_uri	MUST	Identique à celui utilisé dans /auth (vérification BankID)
client_id	MUST	Ton client_id BankID
client_secret	MUST	Ton client_secret BankID (depuis vault, jamais en dur)

4.4 Réponse — tokens reçus

Token	Type	Durée	Usage
access_token	Opaque string	~1 heure	Bearer pour /userinfo et /profile
token_type	String	—	Toujours 'Bearer'
expires_in	Integer	Secondes	Durée de validité access_token
id_token	JWT (JWS ou JWE+JWS)	~1 heure	Identité de base + nonce à vérifier
refresh_token	Opaque string	~1 an	Renouvellement access_token sans ré-auth
scope	String	—	Scopes effectivement accordés par l'utilisateur

4.5 Validation obligatoire de l'id_token

- | | |
|-------------|--|
| MUST | Vérifier la signature JWS de l'id_token avec les clés publiques BankID (JWK). |
| MUST | Vérifier le claim 'nonce' de l'id_token == nonce stocké en session. |
| MUST | Vérifier 'iss' == issuer BankID connu, 'aud' contient ton client_id, 'exp' non expiré. |

4.6 Implémentation

```

@Service
public class BankIdTokenService {

    public TokenResponse exchangeCode(String code, String nonce) {
        MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
        body.add("grant_type", "authorization_code");
        body.add("code", code);
        body.add("redirect_uri", redirectUri);
        body.add("client_id", clientId);
        body.add("client_secret", clientSecret); // Depuis vault

        TokenResponse tokens = bankIdWebClient.post()
            .uri("/token")
            .contentType(MediaType.APPLICATION_FORM_URLENCODED)
            .bodyValue(body)
            .retrieve()
            .onStatus(HttpStatusCode::isError, this::handleTokenError)
            .bodyToMono(TokenResponse.class)
            .block();

        // Validation id_token – MUST
        validateIdToken(tokens.idToken(), nonce);

        return tokens;
    }

    private void validateIdToken(String rawIdToken, String expectedNonce) {
        // Détection : JSON brut ou JWT (JWE ou JWS direct)
        SignedJWT signedJWT = resolveSignedJWT(rawIdToken);

        // Vérification signature avec clé publique BankID
        String kid = signedJWT.getHeader().getKeyID();
        RSAKey bankIdKey = jwkProvider.getSigningKey(kid);
        if (!signedJWT.verify(new RSASSAVerifier(bankIdKey.toRSAPublicKey())))
            throw new JwtValidationException("Signature id_token invalide");

        JWTClaimsSet claims = signedJWT.getJWTClaimsSet();

        // Claims standards – MUST
        if (!bankIdIssuer.equals(claims.getIssuer()))
            throw new JwtValidationException("Issuer invalide");
        if (!claims.getAudience().contains(clientId))
            throw new JwtValidationException("Audience invalide");
        if (claims.getExpirationTime().before(new Date()))
            throw new JwtValidationException("id_token expiré");

        // Nonce – MUST anti-replay
        String receivedNonce = (String) claims.getClaim("nonce");
        if (!MessageDigest.isEqual(
            expectedNonce.getBytes(), receivedNonce.getBytes()))
            throw new JwtValidationException("Nonce invalide – possible replay");
    }
}

```

④

Récupération des données KYC — GET /userinfo

Appel serveur-à-serveur avec `access_token`

5.1 Rôle

Ton Back API appelle /userinfo avec l'`access_token` pour récupérer les données KYC que l'utilisateur a consenties à partager. La réponse peut être du JSON brut OU un JWT (JWS/JWE) selon la configuration de ton application sur le portail BankID.

5.2 Requête HTTP

Élément	Valeur
Méthode	GET
URL	https://oidc.sandbox.bankid.cz/userinfo
Authorization	Bearer {access_token}
Accept	application/json OU application/jwt (si JWS/JWE activé)
TLS	1.3 obligatoire

5.3 Réponse — deux formats possibles

Format	Content-Type	Quand	Action
JSON brut	application/json	JWS/JWE non configuré sur le portail	Parser directement
JWT (JWS)	application/jwt	Signature activée	Vérifier signature JWS avec clé BankID
JWT (JWE+JWS)	application/jwt	Signature + chiffrement activés	Déchiffrer JWE → vérifier JWS

5.4 Claims disponibles (selon scopes accordés)

Claim	Scope	Type	Description
sub	openid	String	Identifiant pairwise unique par application (TOUJOURS présent)
given_name	name	String	Prénom(s)
family_name	name	String	Nom de famille
birthdate	birthdate	String	Date de naissance (YYYY-MM-DD)
age	birthdate	Integer	Âge calculé
majority	birthdate	Boolean	Majeur ou non
addresses	addresses	Array	Liste d'adresses (type, street, city, zip, country)
idcards	idcards	Array	Documents d'identité (type, number, valid_to, country)

pep	legalstatus	Boolean	Personne politiquement exposée
limited_legal_capacity	legalstatus	Boolean	Capacité juridique limitée
verified_claims	verification	Object	Détail de la vérification (trust_framework, niveau)

5.5 Implémentation — détection et traitement

```

@Service
public class BankIdUserInfoService {

    public Map<String, Object> fetchAndProcess(String accessToken) {
        // Appel /userinfo
        ResponseEntity<String> response = bankIdWebClient.get()
            .uri("/userinfo")
            .header(HttpHeaders.AUTHORIZATION, "Bearer " + accessToken)
            .retrieve()
            .toEntity(String.class)
            .block();

        String body = response.getBody();
        String contentType = response.getHeaders()
            .getFirst(HttpHeaders.CONTENT_TYPE);

        // Détection du format – MAY : BankID peut envoyer les deux
        if (contentType != null && contentType.contains("application/jwt")) {
            // JWT reçu – JWS seul ou JWE wrapping JWS
            return jwtProcessingService.processSignedEncryptedJwt(body);
        }
        // JSON brut
        return objectMapper.readValue(body, MAP_TYPE);
    }

    // Après récupération : stocker + auditer
    public String storeKycData(Map<String, Object> claims, String rawPayload) {
        String sub = (String) claims.get("sub");
        String subHash = DigestUtils.sha256Hex(sub);

        // Stockage opérationnel – AES-256-GCM
        encryptionService.encryptAndWrite(subHash,
            objectMapper.writeValueAsString(claims),
            operationalDir);

        // Stockage audit – payload brut (MUST : 15 ans)
        encryptionService.encryptAndWrite(
            subHash + "-" + Instant.now().getEpochSecond(),
            rawPayload,
            auditDir);

        // Fichier meta (non chiffré – pas de PII)
        writeMeta(subHash, sub, claims);

        return subHash;
    }
}

```

⑤

Révocation des tokens — POST /revoke

Fin de session ou expiration

6.1 Rôle

La révocation informe BankID qu'un token n'est plus valide. Elle doit être appelée à la fin de chaque session KYC, et par le batch pour les tokens expirés. Révoquer le refresh_token révoque automatiquement tous les access_tokens associés.

6.2 Requête HTTP

Élément	Valeur
Méthode	POST
URL	https://oidc.sandbox.bankid.cz/revoke
Content-Type	application/x-www-form-urlencoded
TLS	1.3 obligatoire

6.3 Paramètres du body

Paramètre	Obligatoire	Valeur
token	MUST	Le token à révoquer (refresh_token ou access_token)
token_type_hint	SHOULD	'refresh_token' ou 'access_token' (optimise la recherche)
client_id	MUST	Ton client_id
client_secret	MUST	Ton client_secret (depuis vault)

6.4 Stratégie de révocation

- Toujours révoquer le **refresh_token en priorité** — ça révoque tous les access_tokens associés.
- Si le refresh_token est déjà expiré, révoquer l'access_token séparément.
- Une réponse 200 avec body vide = succès. Une réponse 200 même si le token est déjà révoqué (idempotent).

6.5 Implémentation

```
@Service
public class BankIdRevocationService {

    public void revokeSession(String refreshToken, String accessToken) {
        // Révoquer refresh_token en priorité
        if (refreshToken != null) {
            revokeToken(refreshToken, "refresh_token");
        }
        // Révoquer access_token si présent
        if (accessToken != null) {
            revokeToken(accessToken, "access_token");
        }
    }

    private void revokeToken(String token, String hint) {
        MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
        body.add("token", token);
        body.add("token_type_hint", hint);
        body.add("client_id", clientId);
        body.add("client_secret", clientSecret);

        try {
            bankIdWebClient.post()
                .uri("/revoke")
                .contentType(MediaType.APPLICATION_FORM_URLENCODED)
                .bodyValue(body)
                .retrieve()
                .toBodilessEntity()
                .block();
            log.info("Token révoqué : hint={}", hint);
        } catch (Exception e) {
            // Log mais ne pas bloquer – token peut être déjà expiré
            log.warn("Révocation échouée ({}): {}", hint, e.getMessage());
        }
    }
}
```

⑥

Récupération des clés JWK — GET /.well-known/jwks

Au démarrage et sur rotation

7.1 Rôle

Ton Back API récupère les clés publiques de BankID pour vérifier les signatures JWS des id_token, userinfo et profile reçus. Nimbus gère le cache automatiquement et re-fetch sur kid inconnu (rotation de clés).

7.2 Structure des clés BankID

Préfixe kid	Usage (use)	Type	Rôle
*rp-sign / rp-sign	sig	RSA ou EC	Signature des JWT (accompagné d'un certificat QSEAL)
rp-encrypt	enc	EC (P-521)	Chiffrement JWE des réponses vers SeP
Autres	—	—	Clés internes BankID — ignorer

7.3 Rotation des clés — procédure BankID

■ Procédure de rotation (source : security_sep §Key rotation)

1. BankID déploie les nouvelles clés SANS supprimer les anciennes.
2. Les nouvelles clés sont publiées dans le JWKS — durée : ~10 minutes de coexistence.
3. Après 10 minutes, les anciennes clés sont supprimées du JWKS.

Note : les signing keys ne sont PAS supprimées (conservation pour vérification historique).

■ BankID peut effectuer une rotation SANS préavis — le code DOIT gérer le kid inconnu.

7.4 Implémentation avec cache et re-fetch

```
@Component
public class BankIdJwkProvider {

    // Nimbus RemoteJWKSet : cache interne + refresh automatique
    private final RemoteJWKSet<SecurityContext> remoteJwkSet;

    public BankIdJwkProvider(@Value("${bankid.base-url}") String baseUrl)
        throws MalformedURLException {
        URL jwksUrl = new URL(baseUrl + "/.well-known/jwks");
        // timeout connexion 5s, lecture 10s, taille max 50Ko
        this.remoteJwkSet = new RemoteJWKSet<>(jwksUrl,
            new DefaultResourceRetriever(5000, 10000, 50 * 1024));
    }

    public RSAKey getSigningKey(String kid) throws KeySourceException {
        JWKMatcher matcher = new JWKMatcher.Builder()
            .keyID(kid)
            .keyUse(KeyUse.SIGNATURE)
            .build();
        List<JWK> keys = remoteJwkSet.get(new JWKSelector(matcher), null);

        if (keys.isEmpty()) {
            // kid inconnu : forcer un re-fetch (rotation probable)
            log.warn("Kid '{}' inconnu – re-fetch JWK en cours", kid);
            invalidateCache();
            keys = remoteJwkSet.get(new JWKSelector(matcher), null);
            if (keys.isEmpty())
                throw new JwtValidationException("Clé inconnue après re-fetch: " + kid);
        }
        return (RSAKey) keys.get(0);
    }

    public void invalidateCache() {
        // Force Nimbus à re-fetcher au prochain appel
        remoteJwkSet.getJWKSetCache().clear();
    }
}
```


8. Gestion des Erreurs — Tous les Appels

8.1 Codes d'erreur OAuth2 (callback /auth)

Code erreur	HTTP	Cause	Action
access_denied	302	Utilisateur a refusé ou annulé	Page 'authentification refusée'
eid_doesnt_exist	302	Pas de compte BankID éligible	Proposer alternative d'identification
invalid_scope	302	Scope non supporté ou 'openid' manquant	Vérifier config portail
temporarily_unavailable	302	BankID surchargé	Retry avec backoff + message utilisateur
server_error	302	Erreur interne BankID	Log + alerte équipe
invalid_request	302	Paramètre manquant ou invalide	Vérifier construction URL /auth
interaction_required	302	prompt=none impossible	Ne pas utiliser prompt=none pour KYC

8.2 Erreurs /token (HTTP 400/401)

Code erreur	Cause	Action
invalid_grant	Code expiré, déjà utilisé ou redirect_uri différent	Relancer le flux depuis /auth
invalid_client	client_id ou client_secret invalide	Vérifier les credentials du portail
invalid_request	Paramètre manquant dans le body	Vérifier la construction de la requête
unauthorized_client	Application non autorisée pour ce grant	Vérifier config portail

8.3 Handler centralisé

```
@Component
public class BankIdErrorHandler implements ExchangeFilterFunction {

    @Override
    public Mono<ClientResponse> filter(ClientRequest req, ExchangeFunction next) {
        return next.exchange(req).flatMap(response -> {
            if (response.statusCode().is4xxClientError()) {
                return response.bodyToMono(BankIdErrorResponse.class)
                    .flatMap(err -> Mono.error(mapToException(err)));
            }
            if (response.statusCode().is5xxServerError()) {
                return Mono.error(new BankIdUnavailableException(
                    "BankID server error: " + response.statusCode().value()));
            }
            return Mono.just(response);
        });
    }

    private BankIdException mapToException(BankIdErrorResponse err) {
        return switch (err.error()) {
            case "access_denied" -> new UserConsentDeniedException(err.description());
            case "eid_doesnt_exist" -> new UserNotEligibleException(err.description());
            case "invalid_grant" -> new InvalidGrantException(err.description());
            case "temporarily_unavailable" -> new BankIdUnavailableException(err.description());
            default -> new BankIdException(err.error(), err.description());
        };
    }
}
```

9. Synthèse — Sécurité par Appel

Appel	TLS 1.3	State/Nonce	Validation JWT	Cert serveur	Logger
GET /auth	MUST	Générer + stocker	—	Standard FQDN	URL générée (sans state/nonce)
GET /callback	MUST	Vérifier state	—	Standard FQDN	Erreur seulement, jamais le code
POST /token	MUST	Vérifier nonce	id_token MUST	—	Status HTTP uniquement
GET /userinfo	MUST	—	MAY (si JWT)	—	subHash uniquement
POST /revoke	MUST	—	—	—	hint + résultat
GET /jwks	MUST	—	—	—	Rotation détectée

✗ Ce qu'il ne faut JAMAIS logger

Le code d'autorisation reçu au callback.

L'access_token, le refresh_token ou l'id_token en clair.

Les données personnelles (nom, prénom, birthdate...) dans les logs applicatifs.

Le client_secret — il ne doit jamais apparaître dans aucun log ni stack trace.

Uniquement : subHash (SHA-256 du sub), timestamps, status HTTP, codes d'erreur OAuth2.

✓ Certificat standard à la place du QWAC — récapitulatif

Let's Encrypt (gratuit, renouvellement auto 90j) ou DigiCert/Sectigo (payant, 1 an).

FQDN exact : kyc.mondomaine.com — pas de wildcard (MUST NOT).

Clé ECDSA P-256 minimum (préféré) ou RSA 2048 bits minimum.

TLS 1.3 forcé sur la Gateway — interdire explicitement TLS 1.2 et la renégociation.

Rotation annuelle obligatoire (MUST) — automatisable avec Let's Encrypt + ACME.